CSC432 – INFORMATION SECURITY

Dr. Muhammad Sharjeel

muhammadsharjeel@cuilahore.edu.pk

Lecture No 9

Authentication and Message Integrity

Authentication

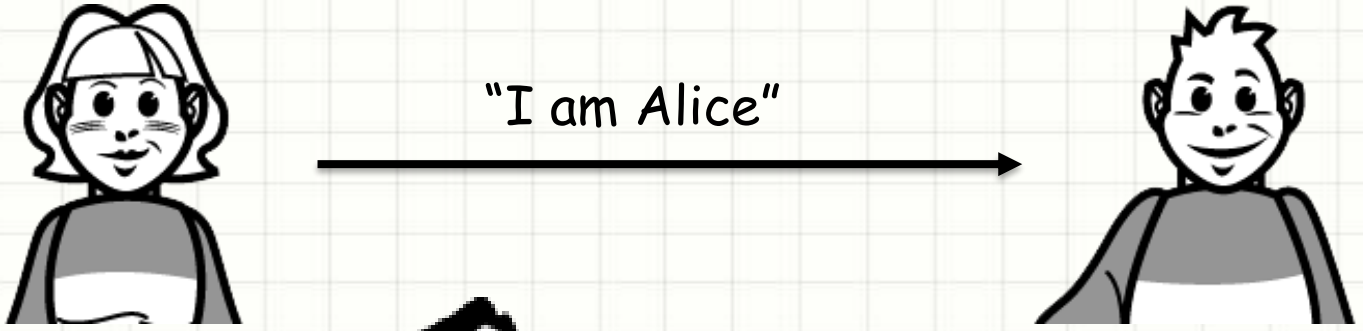
- Authentication is the process of proving one's identity to someone else
- There are three main ways of authenticating an individual:
  - Something you know: A password, cryptographic key, or the correct answer to a challenge-response test
  - Something you own: A physical key, security card or badge, or a one-time password generator
  - Something you are: Some biometric measurement (facial features, fingerprint, retina scan, voice print, and so on)

- **We need an authentication protocol during communication when direct proof cannot be given**

Authentication

A First Attempt

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 1.0: Alice says "I am Alice"

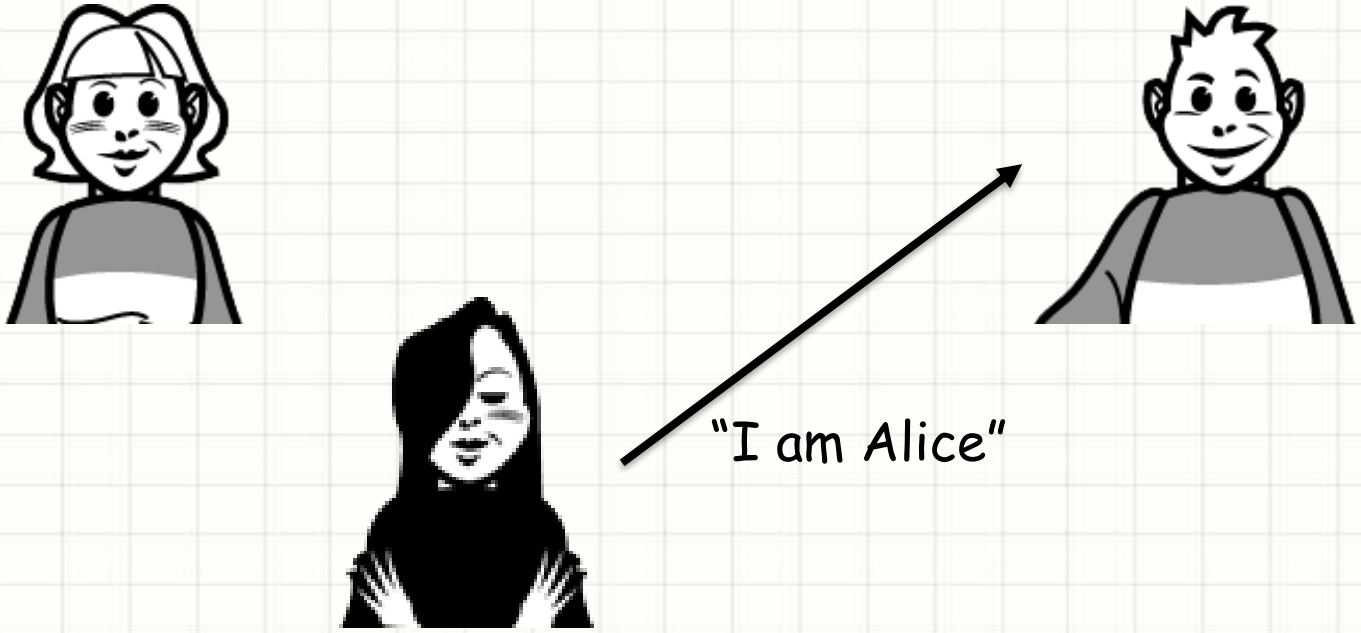


Failure scenario??

Authentication

A First Attempt

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 1.0: Alice says "I am Alice"



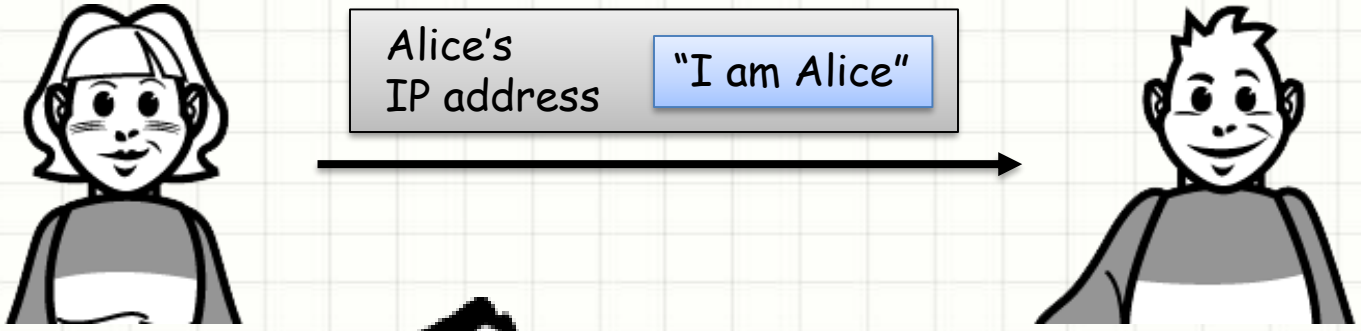
Failure scenario??

On a communication channel, Bob can not "see" Alice, so Trudy simply declares herself to be Alice

Authentication

Another try

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 2.0: Alice says "I am Alice" in an IP packet containing her source IP address

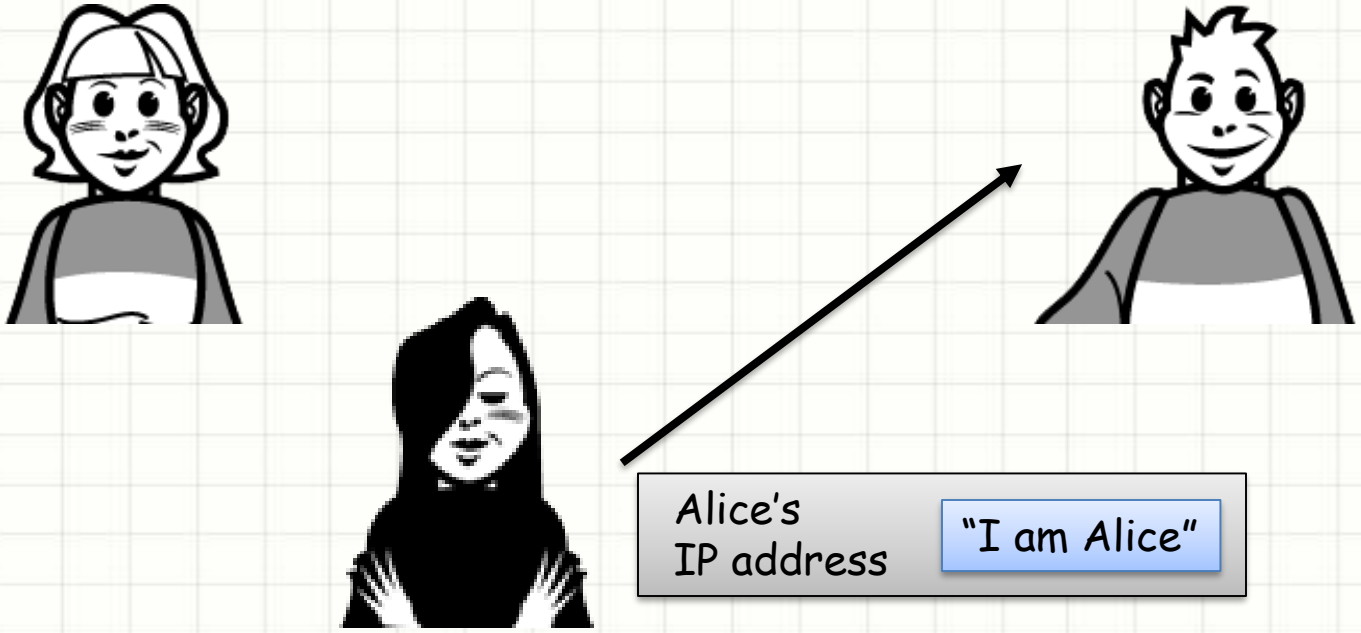


Failure scenario??

Authentication

Another try

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 2.0: Alice says "I am Alice" in an IP packet containing her source IP address



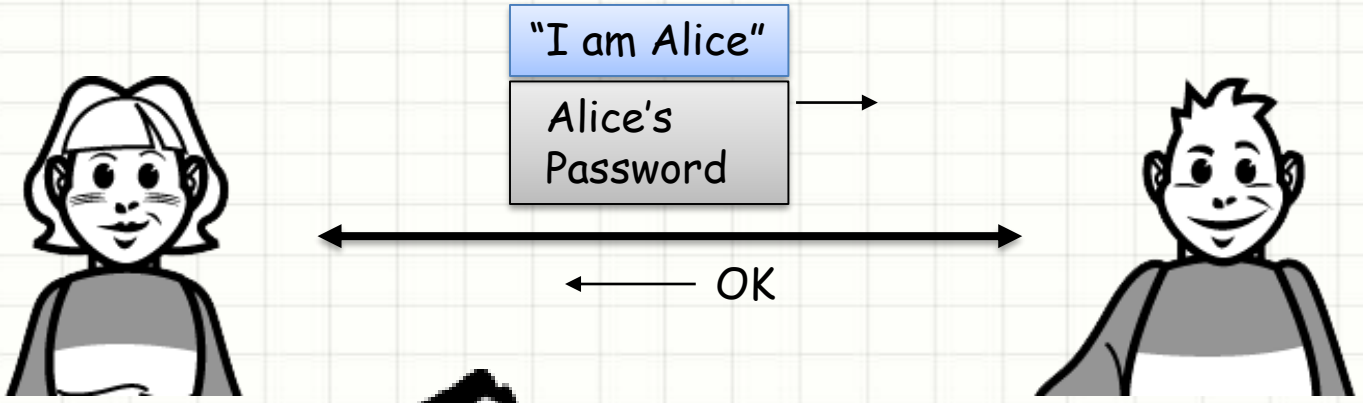
Failure scenario??

Trudy can create a packet "spoofing" Alice's IP address

Authentication

Another try

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 3.0: Alice says "I am Alice" and sends her secret password to "prove" it

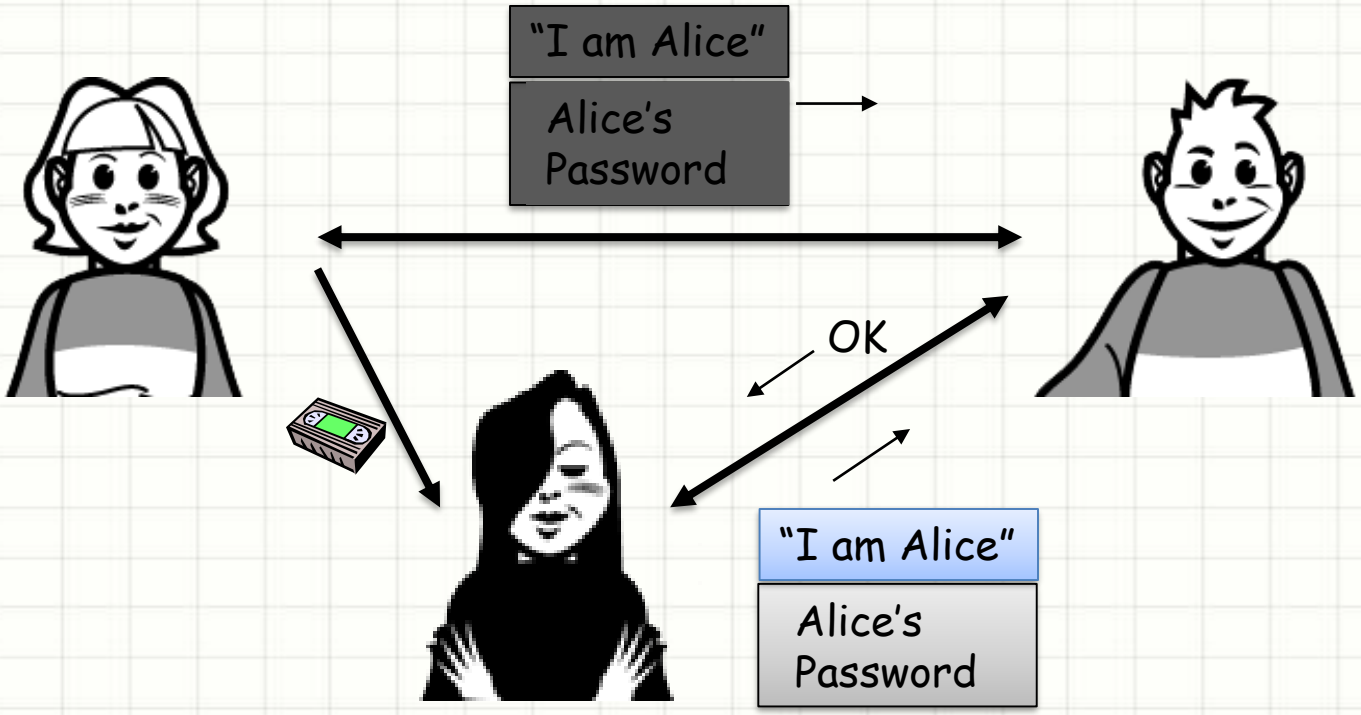


Failure scenario??

Authentication

Another try

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 3.0: Alice says "I am Alice" and sends her secret password to "prove" it



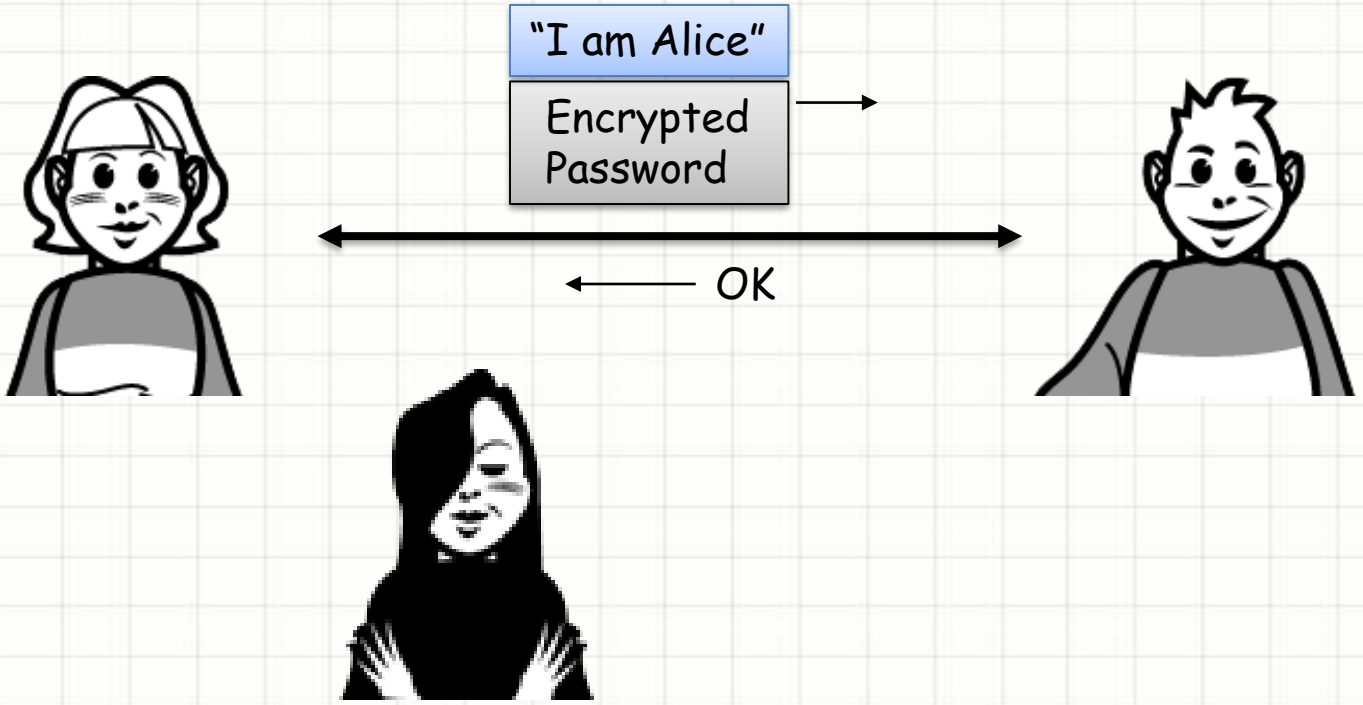
Failure scenario??

Playback attack: Trudy records Alice's packet and later plays it back to Bob

Authentication

Better try

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 3.1: Alice says "I am Alice" and sends her encrypted secret password to "prove" it

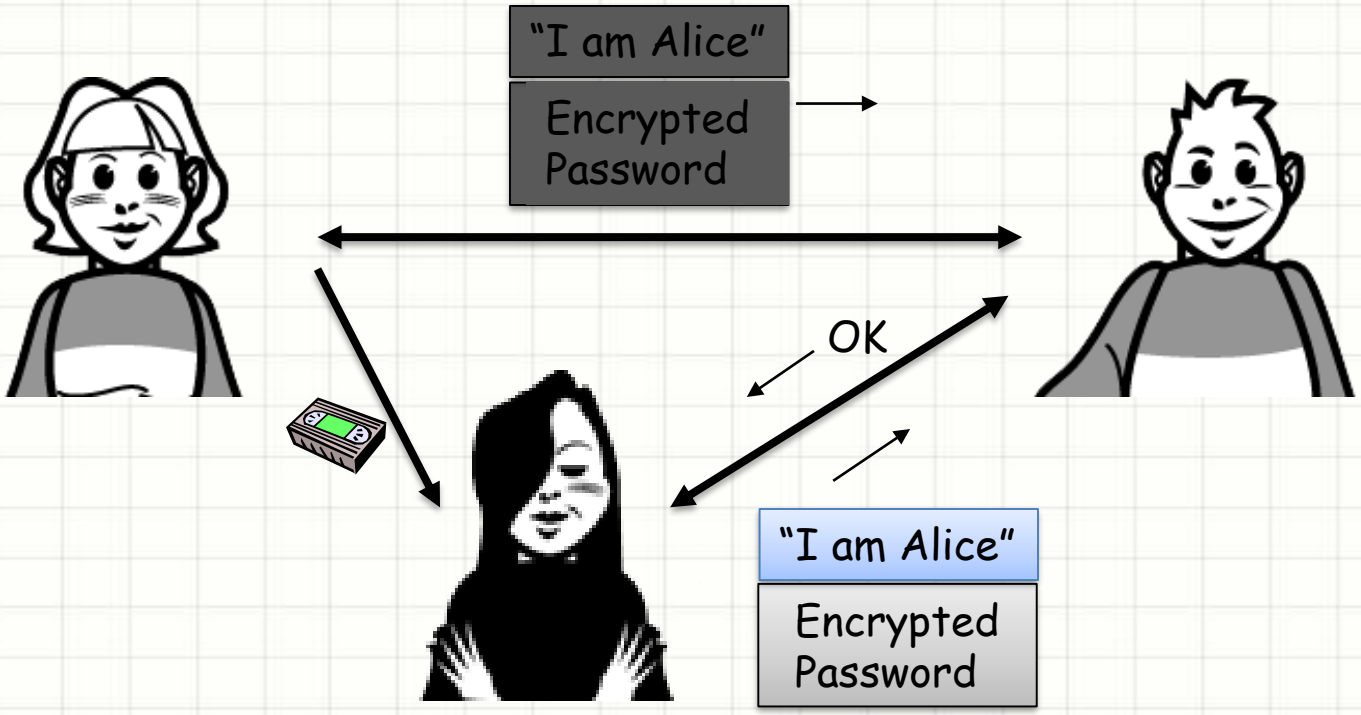


Failure scenario??

Authentication

Better try

- Goal: Bob wants Alice to "prove" her identity to him
- Protocol 3.1: Alice says "I am Alice" and sends her encrypted secret password to "prove" it



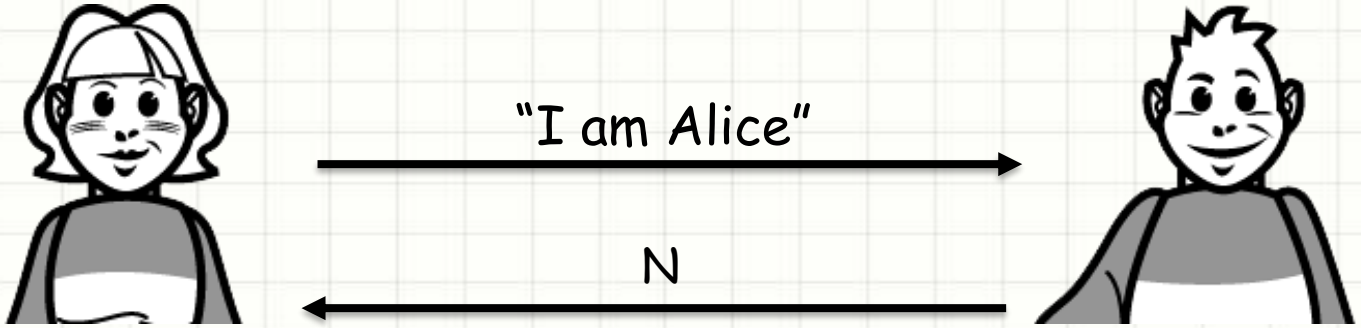
Failure scenario??

Record and playback still works!

Authentication

One More Try

- Goal: Bob wants Alice to "prove" her identity to him
- Nonce: number (timestamp) used only once–in-a-lifetime
- Protocol 4.0: To prove Alice "live", Bob sends Alice nonce, N. Alice must return N, encrypted with shared secret key

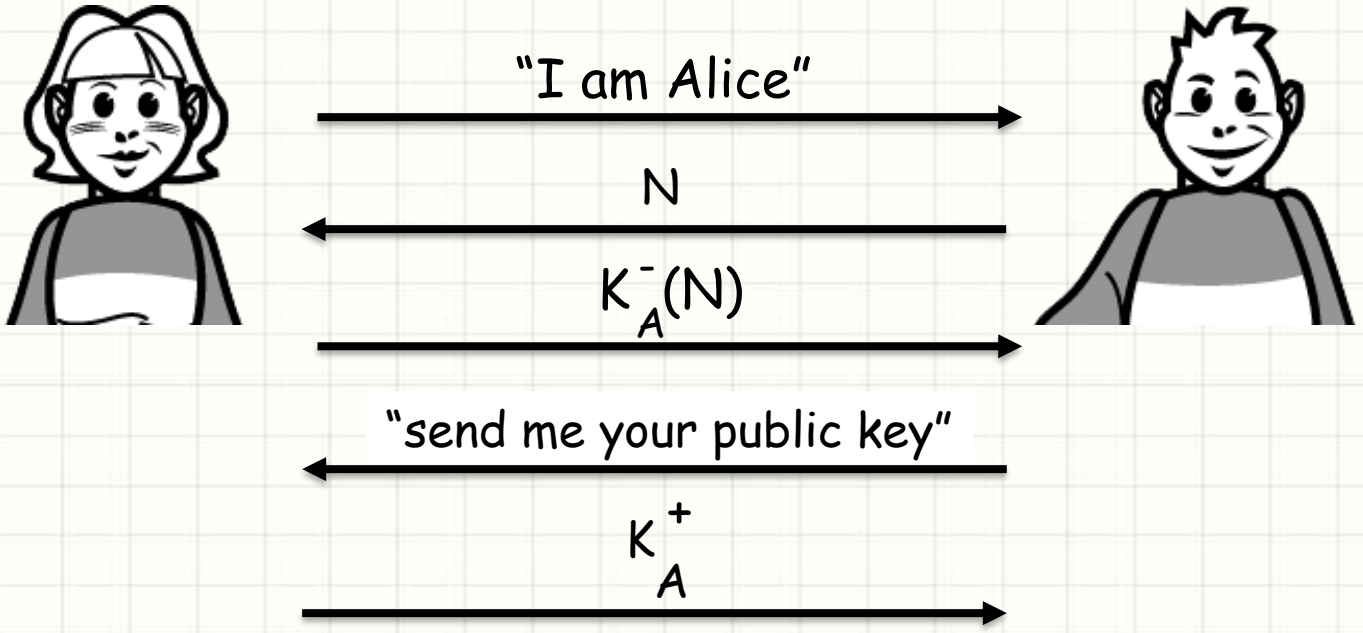


Failures, drawbacks?

Authentication

- Protocol 4.0 requires shared symmetric key
- Can we authenticate using public key techniques?
- Protocol 5.0: use nonce, public key cryptography

Failures, drawbacks?

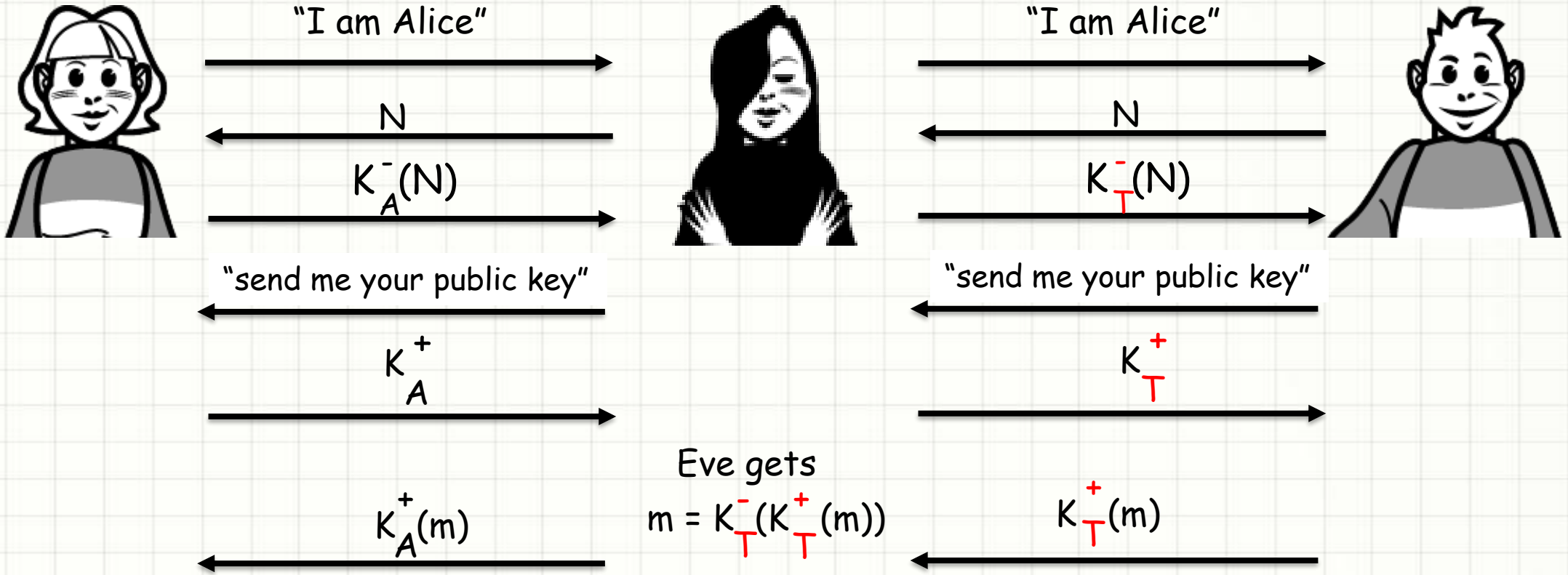


As only Alice has the private key, Bob knows that ONLY she has encrypted N. Bob can read N as,

$$K_A^+(K_A^-(N)) = N$$

Authentication

- Protocol 5.0: Security Hole
- Man in the middle attack: Eve poses as Alice (to Bob) and as Bob (to Alice)



Authentication

Man in the middle attack: Eve poses as Alice (to Bob) and as Bob (to Alice)

Difficult to detect:

- Bob receives everything that Alice sends, and vice versa
- Problem is that Eve receives all messages as well!

Note: The problem was one of key distribution

- If Bob knew or could securely obtain Alice's public key, there would have been no problem!

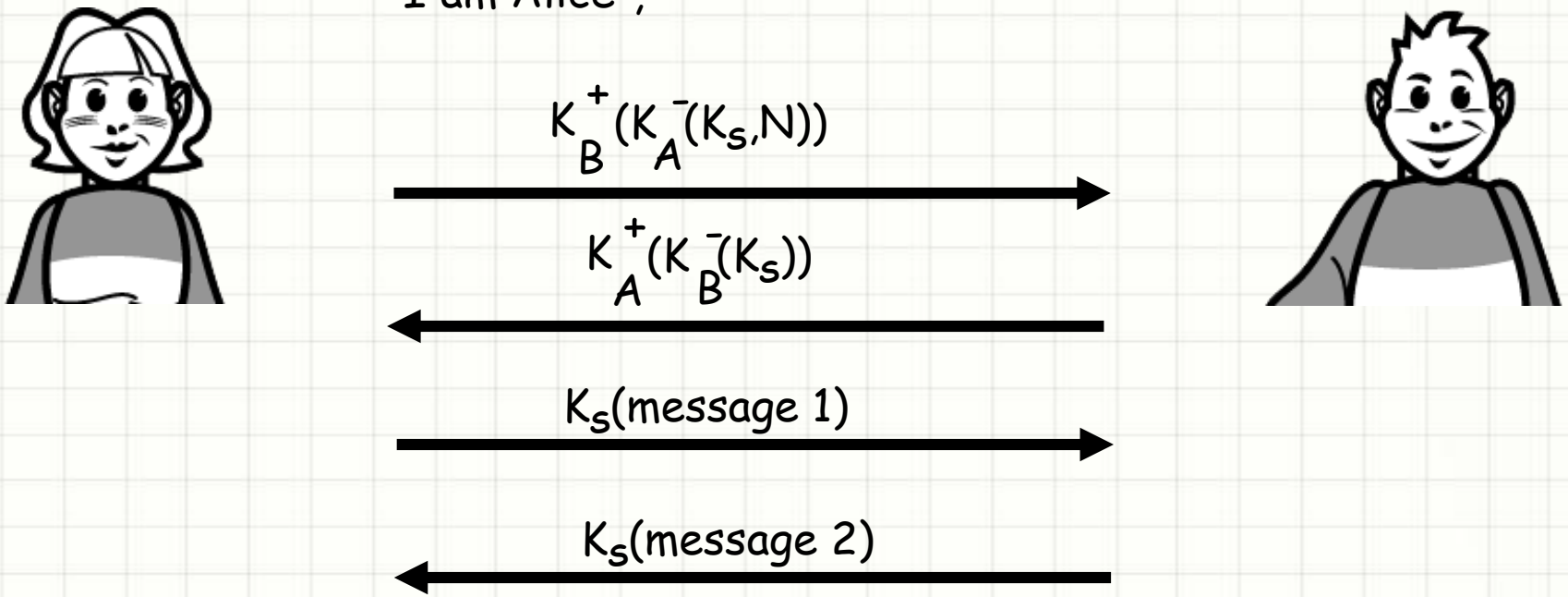
Authentication

Hybrid Secret-Public Key Authentication

- As discussed earlier, the idea is to use public key cryptography to negotiate a symmetric session key for communication
- We must have mutual authentication to ensure the session key can be trusted
- Both Alice and Bob must agree that they are in fact talking to their counterpart
- In this case, we will assume that Alice and Bob already know each other's public keys
- We will look at key distribution and certification very shortly

Authentication

Hybrid Secret-Public Key Authentication



Authentication

Hybrid Secret-Public Key Authentication

- In her message to Bob, Alice identifies herself, and sends a message that only Bob can decrypt with his private key
  - After decrypting the outer message, the inner message can only be decrypted with Alice's public key, proving that she sent it
  - This contains the new session key KS and a nonce to ensure freshness
- Bob sends a response to Alice that only Alice can decrypt with her private key
  - After decrypting the outer message, the inner message can only be decrypted with Bob's public key, proving that he sent it
  - This contains the session key again. If it matches what she sent originally, then they are mutually authenticated and can use the key

Message Integrity

- Bob receives message from Alice, and so Bob wants to ensure:
  - That the message originally came from Alice
  - That the message not changed since sent by Alice

- This is what ensuring message integrity is all about
- There are several ways to go about doing this
- Of course, some are better than others

Message Integrity

Message Checksum

- A checksum is a function calculated over input data to determine if the data has been altered or corrupted
- Checksums are simple to fool, as it is not difficult to alter data in such a way that the same checksum is produced
- Since checksums themselves are not protected, they can be changed after data has been altered anyways
- Checksums, however, are easy to calculate and offer a very basic integrity check

Message Integrity

Message Checksum (Example)

- Checksums used in data packets of the Internet protocol suite are a good example
- They provide some protection against corruption, but as seen below, it is quite easy to have different messages produce the same checksums

message	ASCII format
I O U 1	49 4F 55 31
0 0 . 9	30 30 2E 39
9 B O B	39 42 D2 42
	B2 C1 D2 AC

message	ASCII format
I O U 9	49 4F 55 39
0 0 . 1	30 30 2E 31
9 B O B	39 42 D2 42
	B2 C1 D2 AC

different messages but identical checksums!

Message Integrity

Message Sequence Number

- Another basic integrity check is a sequence number, which gives the position of a message relative to a larger stream of messages
- Sequence numbers can be used to determine if messages have been inserted or deleted, but not if they have been modified
- Since sequence numbers can be forged like checksums, they only provide minimal integrity protection on their own

Message Integrity

Message Digests (MD)

- A special kind of checksum produced using cryptographic means
  - Also referred to as cryptographic checksums
- Typically produced as a hash-code from a one-way function that is difficult to reverse or predict
- Given a message digest *x* and hash function *H*, it should be computationally infeasible to find a message *m* such that H(m) = x
- This function takes the entire input and reduces it to a small value of fixed length (typically 128 to 512 bits in length)

Message Integrity

Message Digests (MD)

- Since modifications to the input data changes the message digest, it is practically impossible to alter a message and still have a matching digest
- However, there is nothing preventing modifying the digest as well to reflect any alterations to the original message

- The solution is to produce a more secure digest, protected by a secret of some kind, referred to as a **message authentication code**

Message Integrity

Message Authentication Codes (MAC)

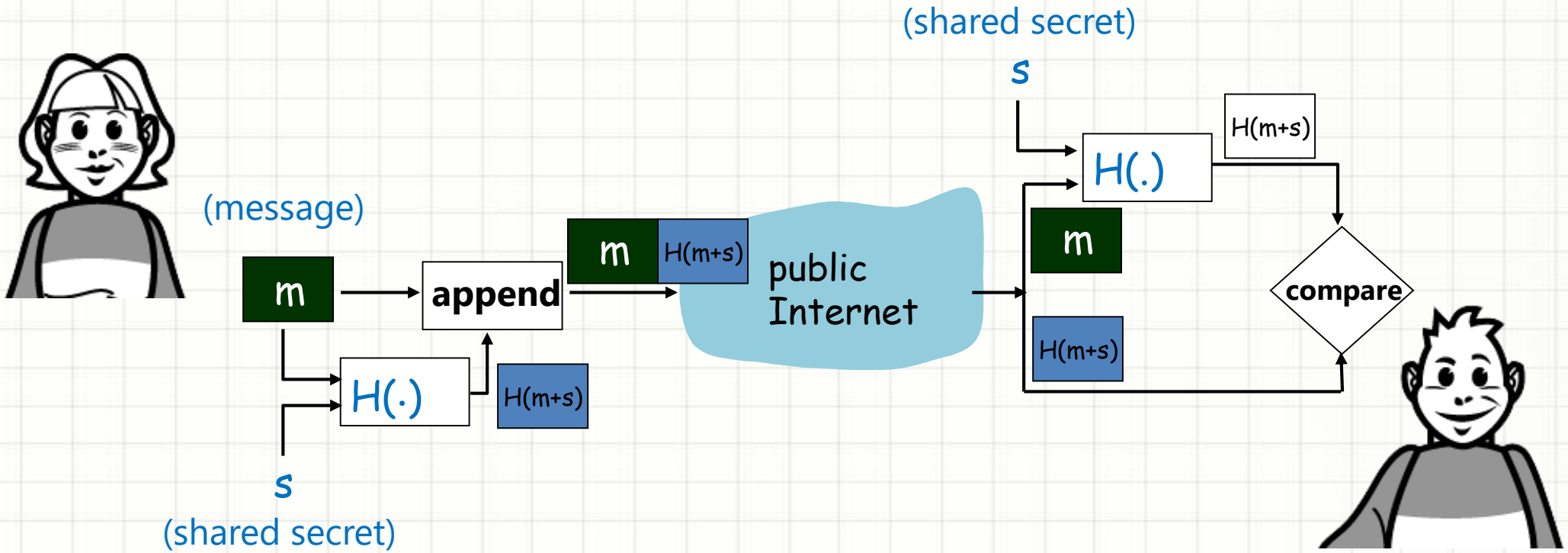
There are a couple of approaches to creating message authentication codes

- One approach is to produce an encrypted digest, by providing a key or password to the digest function
- Without this key or password, modifying the digest itself is no longer an option

- Another approach is to embed a shared secret called an authentication key in with the message before being passed into the digest hash function
- Without knowing this shared secret on the receiving end, the hash function cannot reproduce the same message authentication code

Message Integrity

How Hash Functions Work



Message Integrity

Basic uses of Hash functions

- User authentication
- Message authenticity
- Compact file identifiers
- Used in digital signatures

Message Integrity

Hash Functions

- To be useful for message integrity, a hash function H must have the following properties:
  1. H can be applied to a block of data of any size
  2. H produces a fixed-length output
  3. H(x) is relatively easy to compute for any given x
  4. For any given code h, it is computationally infeasible to find x such that H(x) = h
  5. For any given block x, it is computationally infeasible to find y ≠ x with H(y) = H(x) (referred to as weak collision property)
  6. It is computationally infeasible to find any pair (x, y) such that H(x) = H(y) (referred to as strong collision property)

Message Integrity

- The last property protects against a sophisticated class of attack known as the **birthday attack**

Message Digests: Hash Function Algorithms

- MD5 hash function widely used (RFC 1321)
  - Computes 128-bit message digest in 4-step process
- SHA-1 is also used, US standard [NIST, FIPS PUB 180-1]
  - 160-bit message digest

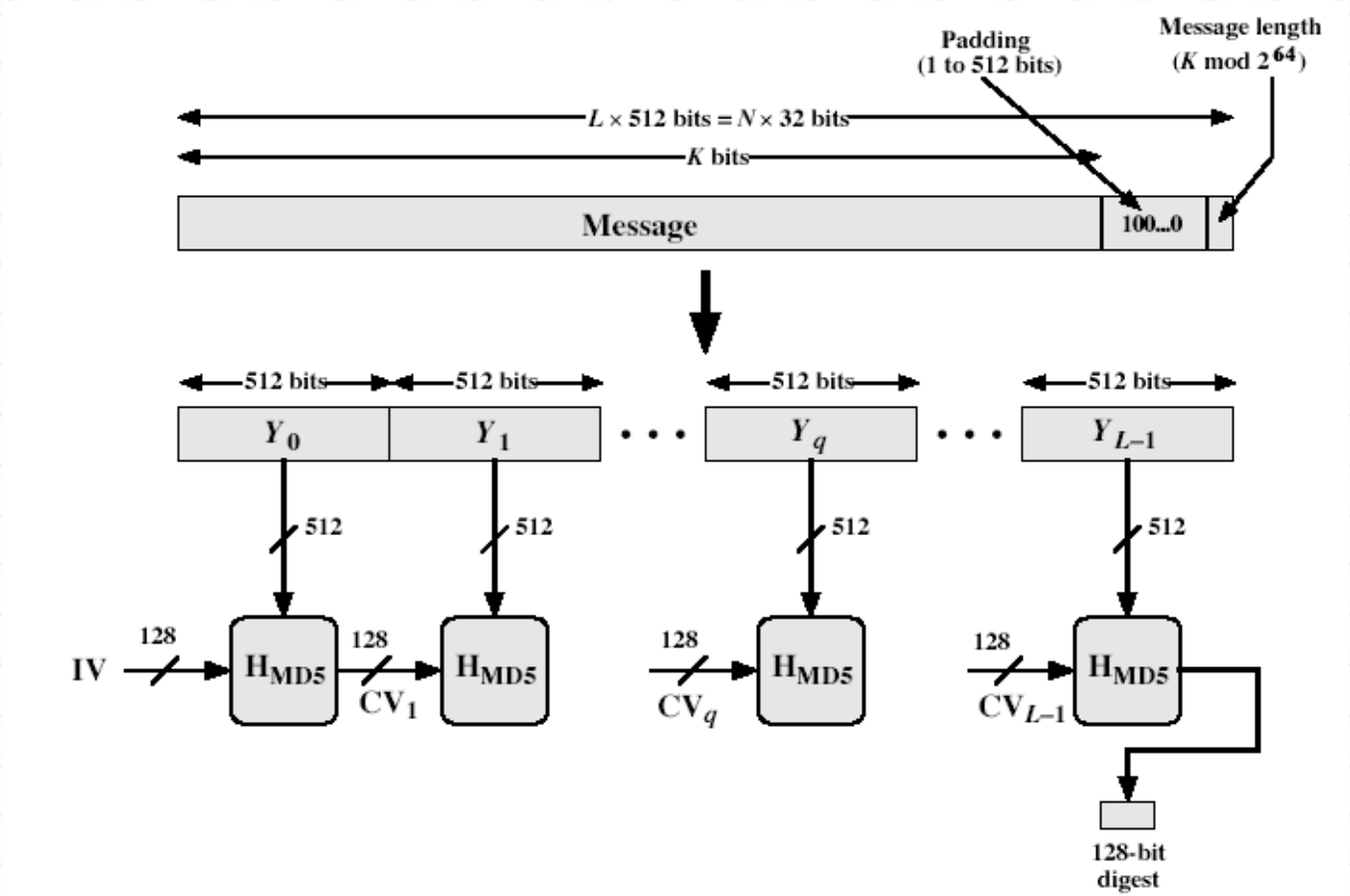
- Issues in both MD5 and SHA-1 have been found in recent years though
- These algorithms are now being phased out more quickly in favor of other, newer approaches like SHA-2, and so on

Message Integrity - Secure Hash Algorithm

- **Secure Hash Algorithm (SHA)** was developed by the National Institute of Standards and Technology (NIST) and published as FIPS 180 in 1993
- Revised as FIPS 180-1 in 1995, generally referred to as SHA-1
- Based on the MD4 algorithm that was developed by Ron Rivest at MIT
- MD4 (and MD5) produce a 128 bit message digest whereas SHA-1 produces a 160 bit
- In 2002, NIST produced a revised version of the standard, FIPS180-2, that defined three new versions of SHA
  - SHA-256, SHA-384, and SHA-512, have hash value lengths of 256, 384, and 512 bits respectively

Message Integrity - Secure Hash Algorithm

- SHA-1 takes as input a message and produces as output a 160 bit message digest, input is processed in 512-bit blocks



Digest generation using MD5 (equally applicable to SHA-1 with 160 bits)

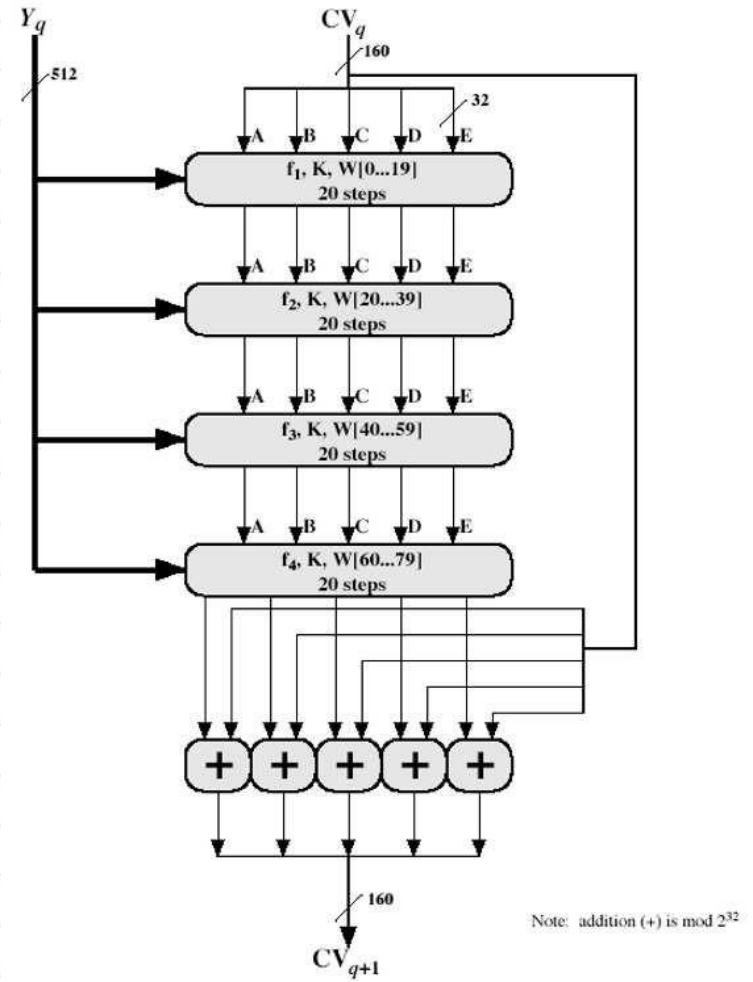
Message Integrity - Secure Hash Algorithm

SHA Overview - overall processing of a message to produce a digest

- Pad message so its length is 448 mod 512
- Append a 64-bit length value to message
- Initialize 5-word (160-bit) buffer (A,B,C,D,E) to
  - (67452301,efcdab89,98badcfe,10325476,c3d2e1f0)
- Process message in 16-word (512-bit) chunks:
  - expand 16 words into 80 words by mixing & shifting
  - use 4 rounds of 20 operations on message block & buffer
  - add output to input to form new buffer value
- Output hash value is the final buffer value

Message Integrity - Secure Hash Algorithm

- SHA-1 compression function (processing of a single 512-bit block)



Message Integrity - Secure Hash Algorithm

SHA-1 verses MD5

- Brute force attack is harder (160 vs 128 bits for MD5)
- Not vulnerable to any known attacks (compared to MD4/5)
- A little slower than MD5 (80 vs 64 steps)
- Both designed as simple and compact

- In 2005, an attack was successful, in which two separate messages deliver the same SHA-1 hash using 2^{69} operations
- NIST announced the intention to phase out SHA-1 and move to a reliance on the other SHA versions by 2010

Message Integrity – The Birthday Attack

- The Birthday attack makes use of what's known as the Birthday Paradox to try to attack cryptographic hash functions

- The birthday paradox can be stated as follows:

"What is the minimum value of *k* such that the probability is greater than 0.5 that at least two people in a group of *k* people have the same birthday?"

- It is a probability problem that deals with the probability of at least one pair of people in a group of *n* people that share the same birthday
- It turns out that the answer is **23 (and not 183)**, which is quite a surprising result

Message Integrity – The Birthday Attack

- In other words, if there are 23 people in a room, the probability that two of them have the same birthday is approximately 0.5
- If there are 100 people (i.e. k=100) then the probability is .9999997, i.e. you are almost guaranteed that there will be a duplicate
- Although this is the case for birthdays we can generalize it for *n* equally likely values (in the case of birthdays n=365)

Message Integrity – The Birthday Attack

- So the problem can be stated like this;

*Given a random variable that is an integer with uniform distribution between **1** and **n** and a selection of **k** instances (**k ≤ n**) of the random variable, what is the probability **P(n, k)**, that there is at least one duplicate?*

It has been found that;

- $K = 1.18\sqrt{n} \approx \sqrt{n}$
- For n = 365, almost equal to 23

Message Integrity – The Birthday Attack

- Now lets look at this in terms of hash codes
- Remember a hash code is a function that takes a variable length message *M* and produces a fixed length message digest
- Assuming the length of the digest is *m* then there are 2^m possible message digests
- However (normally), because the length of *M* will generally be greater than *m* this implies that more than one message will be mapped to the same digest
- The idea is to make it computationally infeasible to find two messages that map to the same digest (strong collision property)

Message Integrity – The Birthday Attack

- If we apply *k* random messages to our hash code what must the value of *k* be so that there is the probability of 0.5 that at least one duplicate (i.e. H(x) = H(y) will occur for some inputs x, y)?
  - This is the same as the question we asked about the birthday duplicates

- Answer is simple;

$k = \sqrt{2^m} = 2^{m/2}$ (as n here is 2^m)

Message Integrity – The Birthday Attack

- If we use a 64-bit hash code then the level of effort required is only on the order of $2^{m/2} = 2^{64/2} = 2^{32}$ which is clearly not sufficient to withstand today's computational systems

Message Integrity – The Birthday Attack

- Given h, attacker computes m, m' such that h(m) = h(m')
  - generate t = $2^{n/2}$ variations on m (all of which convey essentially the same meaning)
  - generate a minor modification m' of m (equal in number as that of m)
  - The two sets of messages (m, m') are compared to find a pair of messages that produce the same hash code (h)
  - repeat the above step until a match is found (this is expected after *t* steps)

- Attacker gives m to the 'source' to hash-then-sign
- consequences: a signature on m is also a valid signature on m'

References

- Wikipedia articles

MD5, http://en.wikipedia.org/wiki/MD5

SHA-1, http://en.wikipedia.org/wiki/SHA-1

- Password Hashing Competition

https://password-hashing.net/

- Hash function online generator

http://www.sha1-online.com/

- "Cryptography and Network Security: Principles and Practice", 6th edition, by William Stallings (chapter 11,12)
- "Network Security: Private Communication in a Public World", by Charlie Kaufman, Radia Perlman, Mike Speciner (chapter 5)

THANKS